OBJECT-ORIENTED PROGRAMMING
MINI PROJECT REPORT

Parcel Tracking

& Delivery Management System

A menu-driven C++ application for booking parcels, generating tracking IDs, assigning delivery agents, calculating charges and maintaining delivery history.

PROJECT SNAPSHOT

Parcel ID Unique tracking code	**Live-style tracking** Status and event history	**C++ OOP** Classes, inheritance, polymorphism

SUBMITTED BY

DETAIL	INFORMATION
Name	Sonu Singh
Student ID	B125122
Branch	CSE
Course	BTech
Academic Year	2026

Prepared as an academic mini-project report for the Object-Oriented Programming course. This document describes the proposed design and implementation plan; sample screens and test cases are illustrative until verified against the completed program.

Declaration

I, **Sonu Singh (Student ID B125122)**, a student of BTech, Branch CSE, hereby submit this report titled **“Parcel Tracking & Delivery Management System”** as an Object-Oriented Programming mini project. The report presents the problem, proposed system design, workflow, object-oriented structure and implementation approach for a C++ application.

The project is intended to be implemented and validated as described. Any sample tracking records, prices, screen layouts and test scenarios in this report are illustrative examples and should be replaced or confirmed using the final working program before submission.

Sonu Singh
B125122 | CSE | BTech | 2026

Acknowledgement

I would like to express my sincere gratitude to my faculty for the guidance and knowledge provided during the Object-Oriented Programming course. The concepts of classes, objects, encapsulation, inheritance, polymorphism and file handling form the foundation of this project.

I also thank my classmates, friends and family for their encouragement and suggestions. Their feedback helped shape the project around a practical problem: keeping parcel booking, assignment and delivery information organised and easy to retrieve.

About this report. The following chapters explain the project idea, system requirements, parcel rules, charge calculation, workflow, flowcharts, class design, implementation plan, sample outputs, testing strategy and future enhancements.

Contents

Section	Page
Declaration and Acknowledgement	2
1. Introduction	3
2. System Overview	4
3. Parcel Data and Delivery Rules	5
4. Delivery Charge Calculation	6
5. Workflow	7
6. Flowcharts	8–9
7. Object-Oriented Concepts and Class Design	10
8. Implementation Plan	11–12
9. Sample Output and Testing Plan	13
10. Conclusion and Future Scope	14
References	14

List of Figures

- Figure 1. Layered system architecture
- Figure 2. Parcel lifecycle
- Figure 3. Charge calculation components
- Figure 4. End-to-end workflow
- Figure 5. Main menu flowchart
- Figure 6. Parcel tracking status flow
- Figure 7. UML-style class diagram

1 Introduction

1.1 Abstract

Parcel Tracking & Delivery Management System is a C++ project designed to model the basic operations of a courier service. It allows a customer to register parcel details, obtain a unique tracking ID, estimate delivery charges and track the parcel through a sequence of delivery stages. An administrator manages delivery agents and assigns parcels, while agents update the status as the parcel moves towards the receiver.

The project demonstrates how object-oriented programming can represent real-world entities such as users, customers, delivery agents, parcels and tracking events. It is planned as a menu-driven console application using C++ classes, inheritance, runtime polymorphism, STL containers and file handling.

Project element	Planned scope
Users	Customer, administrator and delivery agent
Parcel types	Standard and Express
Tracking	Unique ID plus chronological status history
Pricing	Base fee, weight component, distance component and priority fee
Technology	C++11 or later, STL and text-file persistence

1.2 Background and motivation

Courier services handle many parcels at different stages of movement. If booking details, agent assignments and delivery updates are maintained separately, it becomes difficult to answer basic questions such as where a parcel is, who is responsible for it and what amount should be charged. A small centralised system can organise these operations and make parcel information searchable through a tracking number.

1.3 Problem statement

The problem is to design a simple application that records parcel bookings, identifies each parcel uniquely, calculates a transparent delivery charge, assigns an available delivery agent and maintains a traceable sequence of delivery-status updates. The system should validate input and preserve records between program runs.

1.4 Objectives

- Generate a unique tracking ID for every successfully booked parcel.
- Store sender, receiver, parcel weight, origin, destination and delivery type.
- Calculate charges using a clear and configurable pricing formula.
- Maintain delivery-agent records and assign agents to parcels.
- Track parcel status and retain a chronological event history.
- Apply core OOP concepts and use file handling for basic persistence.

2 System Overview

2.1 Main features

- Customer registration and basic customer profile management.
- Parcel booking with sender and receiver information.
- Automatic tracking-ID generation and duplicate-ID prevention.
- Standard or Express delivery selection.
- Charge estimation using weight, distance and priority.
- Admin functions to add agents, view parcels and assign deliveries.
- Agent functions to view assigned parcels and update delivery status.
- Customer tracking lookup by parcel ID, including previous status events.
- Text-file storage for customers, agents, parcels and tracking history.
- Input validation for empty names, invalid weights, invalid menu choices and unsupported status transitions.

2.2 System requirements

Item	Requirement
Language	C++11 or later
Compiler	g++ (GCC), MinGW or a compatible C++ compiler
Operating system	Windows, Linux or macOS terminal
Libraries	iostream, string, vector, fstream, iomanip, sstream, algorithm, ctime
Storage	Text files in a local Data directory
Hardware	Any basic computer capable of compiling and running C++

2.3 Layered architecture

The application can be divided into five logical layers. The menu layer accepts the user's role and command. The input-validation layer checks values before they enter the system. The model layer represents customers, agents, parcels and tracking events. The service layer applies booking, pricing, assignment and tracking rules. The persistence layer reads and writes records to files.

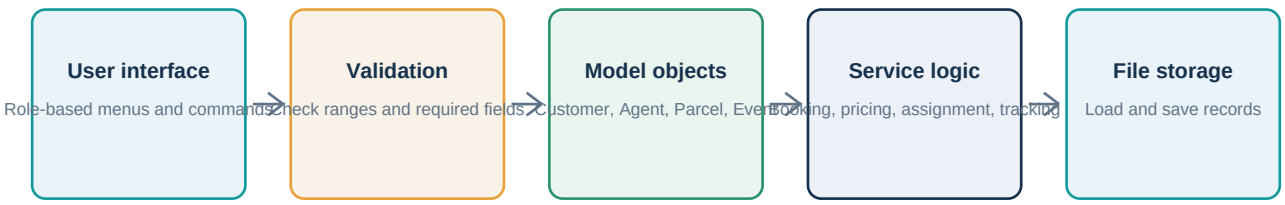


Figure 1: Layered view of the proposed program

3 Parcel Data and Delivery Rules

3.1 Information recorded for a parcel

Field	Purpose / example
Tracking ID	Unique identifier such as PAR20260001
Sender	Name, phone number and pickup address
Receiver	Name, phone number and destination address
Parcel details	Weight in kilograms and optional description
Route	Origin, destination and estimated distance in kilometres
Delivery type	Standard or Express
Assignment	Delivery-agent ID, if assigned
Current status	Booked, Picked Up, In Transit, Out for Delivery, Delivered or delivery exception
Booking date	Date/time at which the parcel was registered

3.2 Tracking status lifecycle

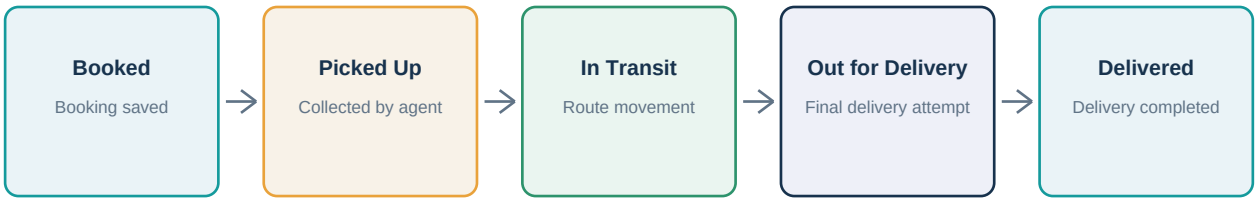
A parcel moves through a controlled sequence. The system should not allow a parcel to jump directly from Booked to Delivered without the intermediate operational updates. An administrator or authorised delivery agent may record an exception such as Delivery Failed or Returned, depending on the rules implemented.

Status	Meaning	Typical next status
Booked	Parcel details and booking are recorded	Picked Up
Picked Up	Parcel collected from sender	In Transit
In Transit	Parcel is moving through the delivery network	Out for Delivery
Out for Delivery	Agent is attempting final delivery	Delivered / Delivery Failed
Delivery Failed	Delivery attempt was unsuccessful	Out for Delivery / Returned
Delivered	Parcel has reached the receiver	Terminal state
Returned	Parcel is being or has been returned	Terminal state

3.3 Business rules and validation

- Tracking IDs must be unique and must not be reused for another parcel.
- Weight must be greater than zero; distance cannot be negative.
- Phone numbers and required names must not be empty.
- Only available agents should receive a new active assignment.
- A delivered parcel should not accept ordinary status updates.
- Every status change should create a new tracking event with a timestamp and location.
- The program should display a clear message when a tracking ID is not found.

Figure 2: Parcel lifecycle


Booked
Booking saved
Picked Up
Collected by agent
In Transit
Route movement
Out for Delivery
Final delivery attempt
Delivered
Delivery completed

4 Delivery Charge Calculation

4.1 Proposed pricing formula

The project uses a transparent formula so that the customer can understand how the delivery amount is calculated. The following rates are sample academic values and can be adjusted by the administrator or changed in the source code.

Total Charge = Base Fee + (Weight × Weight Rate) + (Distance × Distance Rate) + Priority Fee

Component	Illustrative rate	Explanation
Base fee	■50	Fixed booking and handling component
Weight rate	■20 per kg	Increases with parcel weight
Distance rate	■0.10 per km	Increases with route distance
Express priority fee	■50	Added only for Express delivery
Standard priority fee	■0	No additional priority fee

4.2 Worked example

Suppose a customer books a 2.5 kg parcel for a route of 450 km using Express delivery. Applying the illustrative rates:

■50 + (2.5 × ■20) + (450 × ■0.10) + ■50 = **■195**

The amount is an estimate generated by the project formula. A real courier service may apply taxes, fuel surcharges, volumetric weight, remote-area fees or other commercial rules; these are outside the initial project scope.

4.3 Pseudocode

```
function calculateCharge(parcel):
    baseFee = 50
    weightRate = 20
    distanceRate = 0.10

    charge = baseFee
           + parcel.weight * weightRate
           + parcel.distance * distanceRate

    if parcel.type == EXPRESS:
        charge = charge + 50

    return charge
```

4.4 Design note

An abstract Parcel class can declare calculateCharge() as a pure virtual function. StandardParcel and ExpressParcel then provide their own implementations. This demonstrates runtime polymorphism while keeping the pricing decision associated with the parcel type.

5 Workflow

5.1 End-to-end workflow

The main workflow begins when a customer registers or selects the parcel-booking option. After input validation, the system creates a parcel object, generates a unique ID and calculates the estimated charge. The parcel is stored with Booked status. The administrator later assigns an available agent. The agent records pickup and transit updates until the parcel is delivered or an exception is recorded.

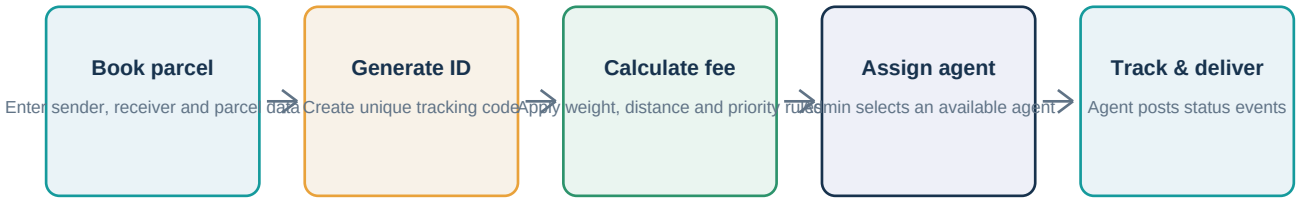


Figure 4: End-to-end parcel workflow

5.2 Step-by-step workflow table

Step	Operation	Responsible module
1	Customer enters sender and receiver details	Customer / Booking
2	System validates required fields and parcel measurements	Input validation
3	System generates a unique tracking ID	ID generator
4	System calculates and displays estimated charge	Pricing service
5	Booking is saved with initial Booked status	Parcel / File storage
6	Admin views unassigned parcels and available agents	Admin dashboard
7	Admin assigns an agent to the parcel	Assignment service
8	Agent updates pickup, transit and delivery status	Agent / Tracking
9	Customer searches tracking ID to view current status and history	Tracking service

5.3 Input checks

- Reject blank sender or receiver names and incomplete addresses.
- Reject non-numeric, zero or negative parcel weights.
- Reject negative distance values and invalid delivery-type choices.
- Check generated IDs against existing parcel records before saving.
- Do not assign an unavailable agent or overwrite an existing assignment without confirmation.
- Validate status transitions and record the event only after a valid update.

6 Flowcharts

6.1 Main program flow

The main program presents a role-based menu. The customer can register and book or track a parcel. The administrator can manage agents and assignments. A delivery agent can view assigned parcels and update their status. Each completed operation returns to the appropriate menu until the user selects Exit.

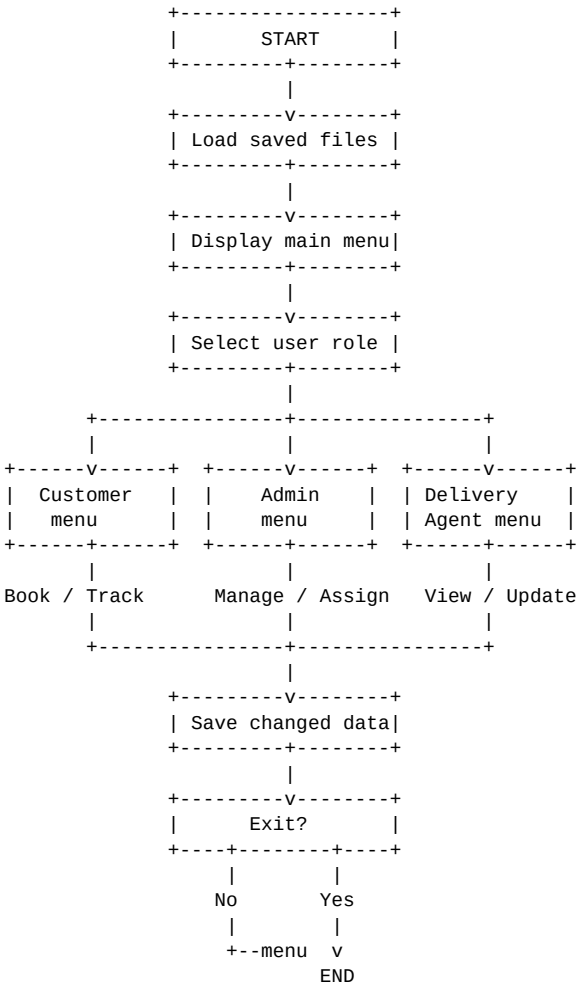


Figure 5: Main program flowchart (text representation)

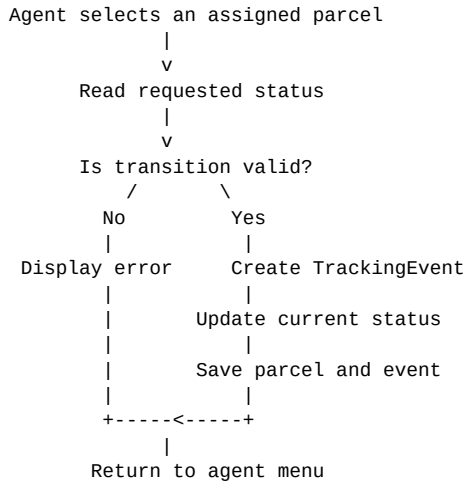
6.2 Booking and tracking flow

```
BOOKING:
Start -> Read parcel details -> Validate input
      -> Generate tracking ID -> Calculate charge
      -> Create Parcel object -> Save parcel
      -> Display receipt -> End

TRACKING:
Start -> Read tracking ID -> Search parcel records
      -> Found?
          No  -> Display 'ID not found' -> End
          Yes -> Display current status
               -> Display chronological tracking events
               -> End
```

Figure 6: Booking and tracking decision flow

6.3 Parcel status update flow



A status update should be treated as a transaction at the application level: validate first, then update the parcel and append the corresponding tracking event. This avoids recording an invalid status in the history.

6.4 Exception paths

Situation	System response
Unknown tracking ID	Display a not-found message and return to tracking menu.
No available agent	Keep parcel unassigned and inform the administrator.
Invalid status transition	Reject the update and show allowed next statuses.
File cannot be opened	Show a storage error; avoid silently discarding existing records.
Duplicate tracking ID	Generate another ID or report an ID-generation failure.

6.5 Persistence sequence

At startup, the application attempts to load customer, agent, parcel and tracking-event records from the Data directory. After a successful operation that changes data, the corresponding records are written back. Each file should use a consistent delimiter or a carefully defined record format so that fields containing spaces are handled safely.

7 Object-Oriented Concepts and Class Design

7.1 OOP concepts used

Concept	Application in this project
Class and object	Customer, DeliveryAgent, Parcel and TrackingEvent represent real entities.
Encapsulation	Keep fields private and expose validated public member functions.
Inheritance	Customer and DeliveryAgent derive from a common User base class.
Abstraction	Parcel defines a common interface for parcel types.
Runtime polymorphism	calculateCharge() is overridden by StandardParcel and ExpressParcel.
Composition	A Parcel owns or contains a collection of its TrackingEvent records.
Constructors	Initialise IDs, names, route data and status safely.
Function overloading	Provide convenient booking or search functions with different parameter lists, if needed.
STL	Use vector for collections and map/unordered_map for lookup by tracking ID.
File handling	Use ifstream and ofstream to load and save records.

7.2 UML-style class diagram

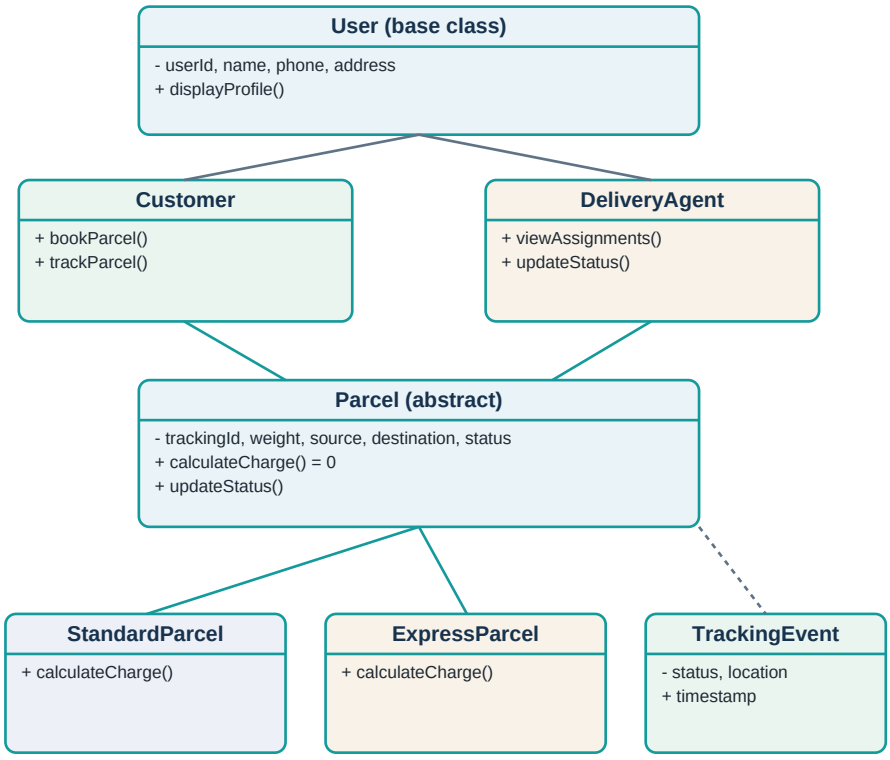


Figure 7: Proposed class relationships (conceptual UML-style diagram)

7.3 Class responsibilities

Class	Responsibilities
User	Common identity and contact details; shared profile display.

Class	Responsibilities
Customer	Customer registration, parcel booking request and tracking lookup.
DeliveryAgent	Agent availability, assigned parcels and status updates.
Parcel	Common parcel data, tracking ID, current status and status-update interface.
StandardParcel / ExpressParcel	Implement delivery-charge calculation for each service type.
TrackingEvent	Store one status change, location, timestamp and optional note.
CourierManagement	Coordinate customers, parcels, agents, assignments and searches.
FileManager	Load and save application records.

8 Implementation Plan

8.1 Suggested project structure

```
ParcelTrackingSystem/
|
|-- main.cpp
|-- User.h / User.cpp
|-- Customer.h / Customer.cpp
|-- DeliveryAgent.h / DeliveryAgent.cpp
|-- Parcel.h / Parcel.cpp
|-- StandardParcel.h
|-- ExpressParcel.h
|-- TrackingEvent.h / TrackingEvent.cpp
|-- CourierManagement.h / CourierManagement.cpp
|-- FileManager.h / FileManager.cpp
|
`-- Data/
    |-- customers.txt
    |-- agents.txt
    |-- parcels.txt
    `-- tracking.txt
```

The project may initially be implemented in fewer files to simplify development. Once the classes work correctly, the code can be separated into header and source files. Each class should have one clear responsibility.

8.2 Core class sketch

```
class Parcel {
protected:
    string trackingId;
    double weight;
    double distance;
    string source;
    string destination;
    string status;

public:
    virtual double calculateCharge() const = 0;
    void updateStatus(const string& newStatus);
    string getTrackingId() const;
    virtual ~Parcel() {}
};

class StandardParcel : public Parcel {
public:
    double calculateCharge() const override;
};

class ExpressParcel : public Parcel {
public:
    double calculateCharge() const override;
};
```

This is a design sketch, not a complete compilable program. Constructors, getters, validation, serialization and error handling must be added during implementation.

8.3 Unique tracking ID generation

A simple academic implementation can use a fixed prefix, the booking year and an incrementing serial number, for example PAR20260001. The next serial value should be loaded from saved data or calculated from existing records so that restarting the program does not accidentally reuse an ID.

```
string generateTrackingId(int year, int serial) {
    string id = "PAR" + to_string(year);
    // Format serial as an eight-digit sequence in the
    // final implementation, or use a fixed-width suffix.
    return id + to_string(serial);
}
```

8.4 File handling approach

- Use ifstream to read saved records when the application starts.
- Use ofstream to write updated records after booking or status changes.
- Keep one record format per file and document the field order.
- Check file-open success and handle missing files as first-run setup.
- For a larger version, replace text files with a database such as SQLite.

8.5 Development roadmap

Phase	Development task	Expected outcome
1	Create classes, constructors and getters/setters	Basic object model
2	Implement customer registration	Customer records
3	Implement booking and ID generation	New parcel can be created
4	Implement charge calculation	Standard and Express quotes
5	Implement agent management and assignment	Parcel linked to an agent
6	Implement status updates and tracking history	Traceable parcel journey
7	Add file loading and saving	Records persist after exit
8	Integrate menus and input validation	Complete console workflow
9	Run test cases and fix defects	Verified application behaviour

8.6 Suggested compile command

```
g++ -std=c++11 main.cpp User.cpp Customer.cpp DeliveryAgent.cpp Parcel.cpp TrackingEvent.cpp CourierManagement.cpp
```

The exact source-file list may change as the project is developed. Compile all required .cpp files together, or use a build configuration if the project grows.

8.7 User interface plan

```
==============================================
   PARCEL TRACKING & DELIVERY MANAGEMENT
==============================================
1. Customer Registration
2. Book a New Parcel
3. Track a Parcel
4. Admin Login
5. Delivery Agent Login
6. Calculate Delivery Charges
7. Exit
Enter your choice:

============= ADMIN DASHBOARD =============
1. Add Delivery Agent
2. View All Parcels
3. Assign Delivery Agent
4. View Unassigned Parcels
5. View Revenue Summary
6. View All Customers
7. Logout
```

9 Sample Output and Testing Plan

9.1 Illustrative booking receipt

```
========== PARCEL BOOKING RECEIPT ==========
Tracking ID      : PAR20260001
Sender           : Rahul Kumar
Receiver         : Aman Singh
Weight           : 2.5 kg
Origin           : Bhubaneswar
Destination      : Patna
Distance         : 450 km
Delivery Type    : Express
Estimated Charge : Rs. 195.00
Current Status   : Booked
============================================
```

The above values are sample data used to demonstrate the intended output format. The final output should be captured from the completed application.

9.2 Illustrative tracking history

Date / time (sample)	Location	Status
02-Nov-2026 09:00	Bhubaneswar	Booked
02-Nov-2026 13:30	Bhubaneswar Hub	Picked Up
03-Nov-2026 06:15	Kolkata Transit Hub	In Transit
03-Nov-2026 18:45	Patna Delivery Hub	Out for Delivery
04-Nov-2026 11:20	Patna	Delivered

9.3 Testing plan

The following cases should be executed after implementation. The expected results are specifications for testing, not recorded test results.

Test ID	Scenario / input	Expected behaviour
T1	Book a valid Standard parcel	Unique ID created; Standard charge shown; status Booked.
T2	Book a valid Express parcel	Express priority fee included.
T3	Enter weight 0 or a negative value	Input rejected and requested again.
T4	Search an existing tracking ID	Current status and tracking history displayed.
T5	Search an unknown tracking ID	Not-found message displayed.
T6	Assign an available agent	Agent ID linked to parcel.
T7	Assign an unavailable agent	Assignment rejected.
T8	Update status from Booked to Picked Up	Status changes and event is recorded.
T9	Attempt invalid status jump	Update rejected with explanation.
T10	Restart application after saving	Previously saved records are loaded.
T11	Mark parcel Delivered, then update again	Ordinary update rejected for terminal parcel.

9.4 Evaluation checklist

- All menu options lead to the correct operation.

- Tracking IDs remain unique across program restarts.
- Charges match the defined formula.
- Only valid agent assignments are accepted.
- Tracking history remains chronological and is not overwritten.
- Saved data can be loaded correctly in a new run.

10 Conclusion and Future Scope

10.1 Conclusion

Parcel Tracking & Delivery Management System is a practical academic project that models the key activities of a small courier service. It connects customers, parcels, delivery agents and tracking events through a common application. The proposed design uses encapsulation to protect data, inheritance to share common user properties, abstraction to define parcel behaviour and polymorphism to calculate charges for different parcel types.

The project also introduces useful programming practices such as input validation, STL-based record management, modular class design and file persistence. Once implemented and tested, it can serve as a foundation for a more complete courier-management application.

10.2 Limitations of the initial version

- The console application does not provide real GPS-based tracking.
- Delivery distance is entered or estimated manually rather than calculated from maps.
- Text files are suitable for a mini project but are not ideal for concurrent multi-user access.
- Authentication in a basic demonstration may be simplified and is not production-grade security.
- The pricing formula is illustrative and does not represent any actual courier company's tariff.

10.3 Future scope

- Add a graphical user interface or web-based customer portal.
- Integrate a database such as SQLite or MySQL.
- Add GPS/map integration for location-based tracking.
- Introduce OTP or signature-based delivery confirmation.
- Add automatic agent allocation based on workload and service area.
- Support notifications through email or SMS.
- Add volumetric weight, insurance, taxes and configurable tariff rules.

References

[1] B. Stroustrup, *The C++ Programming Language*, 4th edition, Addison-Wesley, 2013.

[2] E. Balagurusamy, *Object-Oriented Programming with C++*, McGraw Hill Education.

[3] R. Lafore, *Object-Oriented Programming in C++*, 4th edition, Sams Publishing, 2002.

[4] cppreference.com, C++ standard library reference documentation (string, vector, fstream, iomanip).

[5] Object-Oriented Programming course notes and lecture material.

Parcel Tracking & Delivery Management System | Sonu Singh | B125122 | CSE | BTech | 2026